

PHASE 4 · CI/CD SUPPLY CHAIN

Chapter 15

Harbor

Container Registry & Image Scanning

A secure warehouse for Docker images — every image scanned for vulnerabilities before it ever reaches production.

Estimated time: 3–4 hours

Why Harbor Matters in Supply Chain Security

Harbor is a secure warehouse for container images:

Registry

Store images securely — not on Docker Hub

Audit

Track every image push/pull

Scanning

Detect vulnerabilities in images

Replication

Mirror images across registries

Access Control

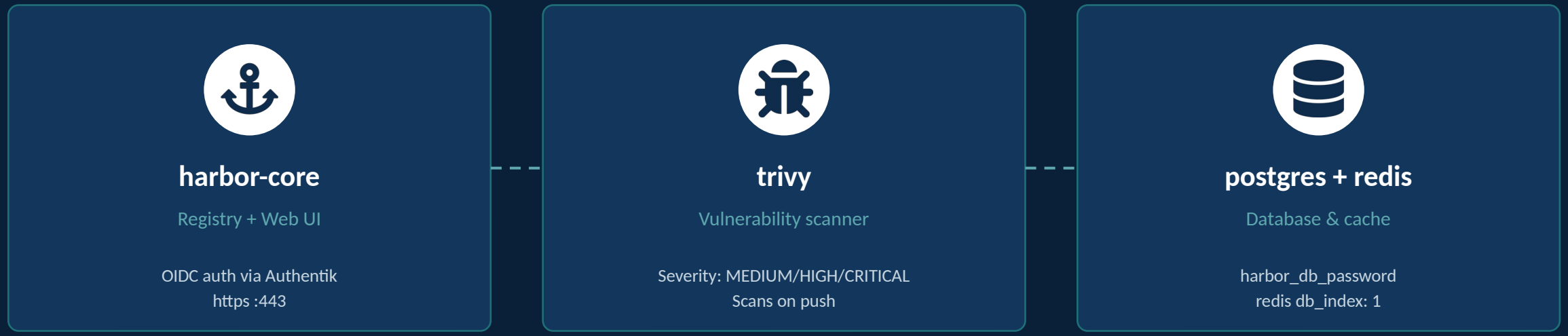
RBAC for who can pull/push images

Signatures

Verify image authenticity

Docker Compose Stack

Deployed on sec-registry-01 (10.0.2.16) from the Harbor offline installer v2.9.0.



Key setup steps

1. Download offline installer → 2. Write harbor.yml → 3. Generate TLS cert → 4. ./prepare && docker-compose up -d

Set Up Project & Access

1

Login to Web UI

https://10.0.2.16
admin / HarborAdminPassword123!

2

Configure OIDC

Administration → Configuration → Authentication. Test via Authentik login

3

Create Project

Projects → New Project
Name: myapp, Visibility: Private

4

Create Robot Account

Administration → Robot Accounts
Project: myapp · push/pull · 90-day expiry

Build, Push, and Authenticate

.gitlab-ci.yml

```
docker_build:
  stage: package
  image: docker:20.10
  services:
    - docker:20.10-dind
  variables:
    REGISTRY_URL: harbor.devsecops.com
    IMAGE_NAME: $REGISTRY_URL/myapp/myapp
  script:
    - docker login -u $HARBOR_USERNAME
        -p $HARBOR_PASSWORD $REGISTRY_URL
    - docker build -t $IMAGE_NAME:$CI_COMMIT_SHA .
    - docker push $IMAGE_NAME:$CI_COMMIT_SHA
    - docker push $IMAGE_NAME:latest
    - docker logout $REGISTRY_URL
  only:
    - main
```

Robot Account Credentials

Project → Settings → CI/CD → Variables

HARBOR_USERNAME

robot account username

HARBOR_PASSWORD

robot account password

Pipeline logs in, builds the image, tags with commit SHA + latest, pushes both, then logs out.

Trivy Scans Every Image, Automatically

Automatic Scanning

- Projects → myapp → Repositories
- Click repository name
- Click image tag
- Vulnerabilities tab shows scan results

```
trivy:

  severity: MEDIUM,HIGH,CRITICAL
  ignore_unfixed: false
  skip_update: false
```

Block Deployment on Vulnerabilities

Option 1 — Prevention Policy

- Projects → myapp → Policy → Vulnerability
- Prevent vulnerable images from being pulled
- Severity threshold: HIGH

Option 2 — Manual Review

- Administration → System Settings
- Require human approval for images with CRITICAL vulnerabilities

myapp:latest Vulnerability Scan

CRITICAL (1)

- CVE-2024-1234: RCE in base image (openssl 1.1.1, fix: 1.1.1w)

HIGH (3)

- CVE-2024-5678: XSS vulnerability
- CVE-2024-9012: Path traversal
- CVE-2024-3456: Information disclosure

MEDIUM (12)

- Various minor vulnerabilities

Action Required: Update base image to patched version, rebuild, re-push.

Content Trust: Verify Authenticity



Enable Content Trust

```
Projects → myapp →
Enable content trust
```



Sign on Push

```
DOCKER_CONTENT_TRUST=1
docker push ...:v1.0
```



Verify on Pull

```
DOCKER_CONTENT_TRUST=1
docker pull ...:v1.0
```

Only images with a valid signature will pull. An unsigned or tampered image is rejected before it ever reaches the node.

Retention Policies & Garbage Collection

Tag Retention

Projects → myapp → Tag Retention

- Keep latest 10 images
- Delete images older than 30 days
- Apply daily

Garbage Collection

Administration → System → Garbage Collection

- Run schedule: daily at 2 AM
- Deletes orphaned blobs
- Reclaims disk space

Pull Images from Harbor

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  template:
    spec:
      containers:
      - name: myapp
        image: harbor.devsecops.com/myapp/myapp:latest
        imagePullPolicy: Always
      imagePullSecrets:
      - name: harbor-secret
```

Secret for Harbor authentication

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: harbor-secret
type: kubernetes.io/dockercfg
data:
  .dockercfg: base64-encoded-
    harbor-credentials
```

The kubelet on each node uses this secret to authenticate to Harbor before pulling the image — no anonymous pulls allowed.

Track Every Action, Mirror Every Image

Audit Logging

Administration → Audit Logs — filter by user, operation, time

14:30:45	**john**	push	`myapp:v1.0`
14:31:10	**gitlab-ci**	push	`myapp:latest`
14:31:15	**k8s-node**	pull	`myapp:latest`

Replication (Optional)

Administration → Replication → Create rule

Source: This Harbor instance

Destination: Other Harbor or Docker Hub

Trigger: On push, daily, or manual

Useful for:

- Disaster recovery
- Geographic distribution
- Public image sharing

Quick Reference

Images won't push

Check docker-compose ps, review logs for errors, verify robot credentials, curl the /api/v2.0/health endpoint.

Scan is slow or stuck

Check Trivy scanner logs, restart with docker-compose restart trivy, confirm free disk space on /opt/harbor/data.

Can't pull from Kubernetes

Verify the harbor-secret exists, decode and inspect its .dockercfg, then test a manual docker login and pull.

Confirming Harbor Is Operational

```
#!/bin/bash
# ~/lab/scripts/validate-chapter-15.sh
set -euo pipefail

HARBOR_IP="10.0.2.16"

# --- Harbor Service ---
ssh sec-registry-01 'docker ps | grep -q harbor'
  ✅ PASS: Harbor container running

# --- Web UI Access ---
curl -s -k https://$HARBOR_IP | grep -qi harbor
  ✅ PASS: Harbor web UI accessible

# --- API Endpoint ---
curl .../api/v2.0/health | grep '"status":"healthy"'
  ✅ PASS: Harbor API healthy

# --- Scanner (Trivy) ---
ssh sec-registry-01 'docker ps | grep -q trivy'
  ✅ PASS: Trivy scanner running

  Passed: 4   Failed: 0
  🎉 Harbor is operational!
```

What the script checks

- Container is running on sec-registry-01
- Web UI responds over HTTPS
- API health endpoint reports healthy
- Trivy scanner container is present

Run after every install/config change to confirm the stack came back up cleanly.

15.14 — CHAPTER SUMMARY

What We Built

- **Harbor registry** — Store Docker images securely
- **Trivy scanning** — Detect vulnerabilities in images
- **OIDC authentication** — Login with Authentik credentials
- **RBAC** — Control who can push/pull images
- **Vulnerability blocking** — Prevent unsafe images from deployment
- **Audit logging** — Track all image operations
- **Image cleanup** — Auto-delete old images

KEY TAKEAWAY

Container Security Is Built-In

BEFORE HARBOR

- Images from unknown sources
- No vulnerability scanning
- No image audit trail
- Manual pull from public registries

AFTER HARBOR

- **All images stored centrally**
- **Automatic vulnerability scanning**
- **Complete audit trail**
- **Only safe images can be deployed**

Next in Phase 4: Ch.16 DefectDojo (vuln management) · Ch.17 Complete CI/CD Pipeline · Ch.18 Supply Chain Security · Ch.19 Compliance & Audit (SLSA, SBOM)